NIMBUS

Guia de instalação, configuração e uso em outros computadores

Edição: Nimbus JSON v4.2
Abrangência: Windows 10/11, Ubuntu/Linux com interface gráfica e macOS
Atualizado em: 21 de outubro de 2026

> **Objetivo:** instalar e utilizar o mesmo projeto Nimbus em outro computador, com ou sem Docker, sem confundir os comandos de Windows, Linux e macOS. Este guia descreve o projeto Python distribuído em ZIP; ele **não é um instalador executável**. O modo gráfico em macOS e em distribuições Linux diferentes de Ubuntu depende da compatibilidade do Python/Tkinter com o sistema.

1. Escolha o modo de funcionamento

Modo	Precisa de Docker?	Onde os dados ficam?	Recomendado para
Local (JSON)	**Não**	`interface/dados/`, no próprio computador	Primeiro teste, uso individual, ambientes sem Docker
Integrado (API + MySQL)	**Sim**	Banco MySQL em volume Docker	Uso com banco de dados e API REST

Em ambos os modos, o arquivo de entrada é JSON com a chave `observacoes`. CSV não é o formato de importação desta versão.

Pré-requisitos gerais: obter o ZIP completo do projeto, conseguir extraí-lo e ter permissão para executar Python. Para o modo integrado também é preciso que o computador ofereça suporte ao Docker, à virtualização necessária e disponha de recursos de memória e disco suficientes. Em máquinas modestas, comece pelo modo local.

2. Preparar a pasta do projeto

1. Copie para o novo computador o ZIP **Nimbus JSON v4.2**.
2. Extraia o conteúdo para uma pasta de sua preferência; não tente executar diretamente dentro do ZIP.
3. Entre na pasta **`analise_dados_meterologicos-main`**.
4. Confirme que nessa pasta existem `interface/`, `projeto_API/`, `shared/`, `docker-compose.yml` e `.env.example`.
5. **Execute todos os comandos a partir dessa pasta**, salvo quando uma etapa pedir explicitamente o contrário.

O nome da pasta externa pode mudar entre computadores. **Não é necessário utilizar o mesmo nome de usuário, caminho do Windows ou senha do computador original.**

PARTE A — Modo local: mais simples, sem Docker

3. Instalar Python e verificar o Tkinter

Windows 10/11 — PowerShell

Instale Python pelo site oficial: **https://www.python.org/downloads/**. Para a interface local, Python 3.13 ou 3.14 é uma opção compatível com os testes deste projeto. Abra **PowerShell**, não Ubuntu nem Git Bash, e execute:

```
python --version
python -m tkinter
```

O segundo comando deve abrir uma pequena janela de teste do Tkinter. Se `python` não for reconhecido, tente `py --version` e `py -m tkinter`.

Na pasta do projeto, instale as dependências locais:

```
python -m pip install -r interface/requirements.txt
```

Se precisar usar py no lugar de python, use py -m pip install -r interface/requirements.txt.

Ubuntu/Linux com ambiente gráfico — Terminal Bash

Em Ubuntu/Debian, instale Python, Tkinter e suporte a ambientes virtuais:

```
sudo apt update
sudo apt install python3 python3-tk python3-venv python3-pip
```

Na pasta do projeto, configure um ambiente virtual e instale as dependências:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r interface/requirements.txt
python -m tkinter
```

A janela do Tkinter exige uma sessão gráfica; um servidor Linux sem ambiente gráfico não abre a interface desktop diretamente.

macOS — Terminal

Instale uma versão compatível de Python do site **https://www.python.org/downloads/macos/**, preferencialmente com o instalador oficial que inclui suporte a Tkinter. Confirme:

```
python3 --version
python3 -m tkinter
```

Na pasta do projeto:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r interface/requirements.txt
```

> **Compatibilidade:** a importação e a interface foram verificadas na série v4.2 em Windows/Linux de teste; a execução específica em macOS precisa ser confirmada no equipamento de destino.

4. Abrir o Nimbus no modo local

Windows — PowerShell:

```
$env:NIMBUS_STORAGE = "local"
python -m interface.app
```

Linux/macOS — Bash com ambiente virtual ativado:

```
NIMBUS_STORAGE=local python -m interface.app
```

Se você usa o **Git Bash no Windows**, a sintaxe das variáveis é a do Bash (export NIMBUS_STORAGE=local), não a do PowerShell ($env:...). No **Ubuntu**, normalmente o executável é python3 quando não há ambiente virtual ativo.

Sinal de sucesso: a janela do Nimbus abre, com indicação de **MODO LOCAL (JSON)**. Nesse modo, não é necessário configurar MySQL, criar .env ou iniciar Docker.

5. Importar dados meteorológicos em JSON

Na interface, entre em **Importação JSON** → **Procurar arquivo JSON** → **Validar e importar JSON**. Depois, consulte **Pontos**, **Fontes**, **Registros**, **Dashboard** e **Análises**.

Exemplo válido de arquivo exemplo_nimbus.json:

```
{
  "observacoes": [
    {
      "ponto": {
        "nome": "ESTACAO DEMONSTRACAO",
        "latitude": -22.90,
        "longitude": -43.20,
        "cidade": "Rio de Janeiro",
        "estado": "RJ"
      },
      "fonte": {
        "nome": "FONTE FICTICIA",
        "tipo": "SIMULADO"
      },
      "data_hora": "2025-10-18T14:00:00",
      "temperatura": 23.50,
      "umidade": 72.00,
      "precipitacao": 0.50
    }
  ]
}
```

Regras principais: `ponto` exige nome e coordenadas; cidade/estado são opcionais. `fonte` exige nome e tipo. Datas devem estar em ISO 8601, sem offset, no horário civil de São Paulo; datas futuras são rejeitadas. Temperatura, umidade e precipitação precisam obedecer aos limites e à precisão configurados no Nimbus. Reimportações são identificadas como **duplicadas** e não criam novos registros.

PARTE B — Modo integrado: Docker, API Flask e MySQL

6. Instalar o Docker conforme o sistema

Windows — caminho recomendado para computadores novos: instale o **Docker Desktop** com backend **WSL 2** e inicie o aplicativo. Siga os requisitos da página oficial: https://docs.docker.com/desktop/setup/install/windows-install/. O comando `wsl --install`, executado em PowerShell como administrador, instala o WSL em sistemas compatíveis; reinicie se solicitado. Não instale outro Docker Engine dentro da mesma distribuição WSL sem um motivo específico: escolha um único caminho para evitar conflitos.

Windows — alternativa já usada no projeto original: instalar **Docker Engine dentro do Ubuntu/WSL**, sem Docker Desktop. Nesse caso, os comandos Docker funcionam **no Ubuntu**; eles podem continuar indisponíveis no PowerShell do Windows. Consulte https://docs.docker.com/engine/install/ubuntu/. Se necessário:

```
sudo systemctl start docker
sudo docker compose version
```

Ubuntu/Linux: instale o Docker Engine e o plugin Docker Compose seguindo a documentação oficial de sua distribuição; no Ubuntu: https://docs.docker.com/engine/install/ubuntu/. Confirme `docker compose version` e `docker info` (use `sudo` se a permissão do usuário ainda não estiver configurada).

macOS: instale e abra **Docker Desktop for Mac** correspondente ao seu processador (Apple Silicon ou Intel): https://docs.docker.com/desktop/setup/install/mac-install/. Confirme `docker compose version` no Terminal.

7. Preparar o arquivo .env com senhas próprias

Faça os comandos **na mesma pasta que contém `docker-compose.yml`**.

Windows — PowerShell:

```
Copy-Item .env.example .env
notepad .env
```

Ubuntu/Linux/macOS — Bash:

```
cp -n .env.example .env
nano .env
```

No `.env`, os campos relevantes devem ter esta estrutura (os textos em maiúsculas são **placeholders**, substitua-os por senhas reais diferentes):

```
DB_HOST=127.0.0.1
DB_PORT=3306
DB_NAME=nimbus
DB_USER=nimbus_user
DB_PASSWORD=SUBSTITUA_POR_SENHA_FORTE_DO_USUARIO
MYSQL_ROOT_PASSWORD=SUBSTITUA_POR_OUTRA_SENHA_FORTE
NIMBUS_STORAGE=api
NIMBUS_API_URL=http://127.0.0.1:5000
```

Não compartilhe esse arquivo e não o inclua em ZIPs, e-mails ou repositórios. A senha de sudo do Ubuntu não precisa ser a mesma senha do MySQL. Dentro dos containers, o Compose utiliza o nome de serviço banco para a conexão com MySQL, independentemente do DB_HOST local.

8. Iniciar os containers

Execute **onde o Docker funciona**:

```
docker compose config --quiet
docker compose up --build -d
docker compose ps
```

Os espaços antes de docker são apenas formatação; no terminal digite normalmente docker compose .... **Se você instalou Docker Engine no Ubuntu/WSL e precisa de permissão de administrador, utilize sudo docker compose ... nos três comandos.** No Docker Desktop funcionando no Windows/macOS, normalmente não é necessário sudo.

Resultado esperado: o container banco aparece como **healthy** e o container api como **running/up**.

Verifique a saúde da API:

Windows — PowerShell:

```
curl.exe http://127.0.0.1:5000/health
```

Ubuntu/Linux/macOS — Bash:

```
curl http://127.0.0.1:5000/health
```

Resposta de referência:

```
{"banco":"conectado","status":"ok"}
```

A resposta confirma conectividade API ↔ banco naquele momento. Ela **não** substitui o teste de importação JSON.

9. Abrir a interface no modo API

Deixe os containers ligados.

Windows — PowerShell (mesmo quando o Docker roda no WSL):

```
$env:NIMBUS_STORAGE = "api"
$env:NIMBUS_API_URL = "http://127.0.0.1:5000"
python -m interface.app
```

Ubuntu/Linux/macOS — Bash:

```
export NIMBUS_STORAGE=api
export NIMBUS_API_URL=http://127.0.0.1:5000
python -m interface.app
```

Se não houver ambiente virtual ativo no Linux/macOS, use python3 e garanta que as dependências estejam instaladas. **Sinal de sucesso:** indicador **API / MySQL**. Importe o JSON pela tela e confira se os dados aparecem nos cadastros e no dashboard. A comunicação HTTP completa deve ser verificada em cada computador.

10. Parar e reiniciar sem apagar os dados

```
docker compose stop       # suspende os serviços
docker compose start      # retoma os serviços
docker compose ps         # mostra o estado
docker compose logs --tail=100 api
```

Se o Ubuntu exigir privilégios, prefixe os comandos com sudo. **Evite docker compose down -v: a opção -v remove volumes e pode apagar definitivamente o banco MySQL.** docker compose down sem -v remove containers e rede, mas preserva os volumes nomeados.

PARTE C — Transferência, backup e segurança

11. Levar o Nimbus para outro computador sem perder os dados

Se utiliza o modo local (JSON): feche a interface; faça uma cópia de segurança da pasta **interface/dados/**; copie o ZIP do projeto e essa pasta para o novo computador; execute em modo local. Não substitua uma pasta dados que já contenha registros sem antes fazer backup de ambas.

Se utiliza API/MySQL: o ZIP do programa **não contém os dados do volume Docker**. É necessário fazer um backup lógico do MySQL no computador de origem e restaurá-lo no destino, além de copiar os arquivos do projeto. Para usuários experientes no terminal Bash do Ubuntu/Linux/macOS, com os containers em execução, um exemplo de exportação é:

```
docker compose exec -T banco sh -c \
  'exec mysqldump -u "$MYSQL_USER" -p"$MYSQL_PASSWORD" "$MYSQL_DATABASE"' \
  > backup_nimbus.sql
```

Se necessário, prefixe docker compose com sudo. Verifique se o arquivo SQL foi criado e contém as tabelas antes de considerá-lo um backup válido. **No Windows PowerShell, não copie cegamente o comando Bash de redirecionamento:** use o terminal Ubuntu/WSL ou uma rotina de backup validada para seu ambiente. A restauração deve ser feita em banco compatível e após preservar o banco de destino.

Os dados locais JSON e os dados do MySQL são **dois armazenamentos diferentes**. Alterar NIMBUS_STORAGE não migra informações automaticamente.

12. Segurança e limites de utilização

- Defina senhas diferentes para DB_PASSWORD e MYSQL_ROOT_PASSWORD; guarde-as em um gerenciador de senhas.
- Não envie o .env junto com o projeto. Um .env.example sem credenciais reais pode ser distribuído.
- Mantenha o Docker e o Python atualizados dentro das versões compatíveis com a aplicação.
- A API desta versão foi projetada para acesso local: **não publique as portas na internet** sem autenticação, TLS e controles de acesso.
- Evite instalar pacotes aleatórios ou executar comandos obtidos de fontes não confiáveis como administrador.

PARTE D — Resolução dos erros mais comuns

Mensagem / situação	Causa provável	Providência
docker: command not found	Docker ausente **nesse terminal**	Identifique se o Docker foi instalado no Windows (Desktop) ou Ubuntu/WSL (Engine). Execute no terminal correto.
:NIMBUS_STORAGE: command not found	Comando PowerShell usado no Bash	Use export NIMBUS_STORAGE=api no Bash ou $env:NIMBUS_STORAGE = "api" no PowerShell.
python: command not found no Ubuntu	Comando costuma ser python3	Use python3 ou ative .venv.
No module named tkinter	Tcl/Tk não instalado na distribuição Linux	No Ubuntu/Debian, instale python3-tk.
required variable DB_PASSWORD is missing	.env ausente/incompleto	Crie o .env ao lado do Compose e informe DB_PASSWORD e MYSQL_ROOT_PASSWORD.
sudo: Authentication failed	Senha do usuário Ubuntu incorreta	Use a senha do usuário Linux, não a do

Mensagem / situação	Causa provável	Providência
		Windows nem do MySQL.
permission denied ... docker.sock	Usuário Linux sem autorização Docker	Teste sudo docker compose ...; revise permissões de forma segura.
connection refused ao abrir API	Flask fora do ar, porta incorreta ou Docker parado	Teste /health, docker compose ps e docker compose logs --tail=100 api.
NoneType ... configure ao clicar em importar	Versão antiga com erro no botão Tkinter	Use os arquivos corrigidos da versão **v4.2**.
0 importadas, 140 duplicadas	Arquivo já importado	Comportamento correto: não duplica registros.
Porta 5000 ou 3306 ocupada	Outro serviço utiliza a porta	Identifique o serviço antes de mudar mapeamentos; não apague containers ou dados sem backup.
API abre, mas GUI não conecta	NIMBUS_API_URL incorreta ou firewall/rede	Teste curl.exe http://127.0.0.1:5000/health no **Windows** onde a GUI roda.

13. Checklist antes de considerar a instalação concluída

1. python --version ou python3 --version funciona e python -m tkinter abre a janela de teste.
2. O projeto foi extraído por completo e o terminal está em analise_dados_meterologicos-main.
3. No **modo local**, a interface abre e um JSON válido é importado.
4. No **modo API**, a configuração Docker é válida; MySQL está healthy; API está running; /health mostra banco conectado.
5. A interface apresenta o modo esperado (**MODO LOCAL (JSON)** ou **API / MySQL**).
6. Uma importação é confirmada em **Registros** e no **Dashboard**; a segunda importação é reconhecida como duplicada.
7. Existe um backup apropriado ao modo escolhido.

Links oficiais

- Python: https://www.python.org/downloads/
- Tkinter: https://docs.python.org/3/library/tkinter.html
- WSL no Windows: https://learn.microsoft.com/pt-br/windows/wsl/install/
- Docker Desktop no Windows: https://docs.docker.com/desktop/setup/install/windows-install/
- Docker Engine no Ubuntu: https://docs.docker.com/engine/install/ubuntu/
- Docker Desktop no macOS: https://docs.docker.com/desktop/setup/install/mac-install/

Nota de escopo: o procedimento foi preparado com base na arquitetura e nos comandos da versão Nimbus JSON v4.2 e no ambiente Windows/WSL relatado. Ele não constitui certificação de execução em todo sistema ou configuração de hardware; qualquer instalação nova deve passar pelo checklist final.